Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Cyber Sécurité — TP7 partie II — Cryptographie
3e année Cybersécurité — École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiants :	HATHOUTI Mohammed Taha JIDAL Ilyas KABORE Mohammad Sharif Jonathan
Filière :	Cybersécurité
Année :	2025/2026
Enseignant :	M.SEBBAR & Mme.GADI
Date :	18 avril 2026

Table des matières

Objectifs **2**

1 Exercice 1 — Mode ECB (*Electronic Codebook*) **3**
1.1 Présentation du mode ECB 3
1.2 Question 1 — Fonctionnement du mode ECB (pseudo-code) 3
1.3 Question 2 — Découpage en blocs 3
1.4 Question 3 & 4 — Chiffrement ECB et texte chiffré final 3
1.5 Question 5 — Déchiffrement et vérification 4
1.6 Question 6 — Redondance dans le texte chiffré 4
1.7 Question 7 — Permutation de l'ordre des blocs chiffrés 4
1.8 Question 8 — Bilan de sécurité du mode ECB 4

2 Exercice 2 — Mode CBC (*Cipher Block Chaining*) **5**
2.1 Présentation du mode CBC 5
2.2 Question 1 — Fonctionnement du mode CBC (pseudo-code) 5
2.3 Question 2 — Découpage en blocs 5
2.4 Question 3 & 4 — Chiffrement CBC et texte chiffré final 5
2.5 Question 5 — Déchiffrement et vérification 6
2.6 Question 6 — Redondance dans le texte chiffré 6
2.7 Question 7 — Permutation de l'ordre des blocs chiffrés 6
2.8 Question 8 — Bilan de sécurité du mode CBC 6
2.9 Question 9 — Propagation d'erreur 6
2.10 Question 10 — Applications de CBC 7

3 Exercice 3 — Schéma de Feistel (2 tours) **7**
3.1 Principe du schéma de Feistel 7
3.2 Algorithme implémenté (Python) 7
3.3 Code Python 8
3.4 Exemple d'exécution 11
3.5 Analyse du schéma de Feistel 11

Conclusion **12**

Objectifs

L'objectif de ce TP est d'étudier et d'implémenter les principaux modes opératoires des chiffrements par blocs, ainsi que le schéma de Feistel. Les points abordés sont les suivants :

- Mode ECB (*Electronic Codebook*) : chiffrement, déchiffrement et analyse de sécurité ;
- Mode CBC (*Cipher Block Chaining*) : chiffrement, déchiffrement, propagation d'erreurs et comparaison avec ECB ;
- Implémentation d'un algorithme de chiffrement basé sur le schéma de Feistel (2 tours) en Python.

Toutes les implémentations sont réalisées en Python (Jupyter Notebook).

1 Exercice 1 — Mode ECB (*Electronic Codebook*)

1.1 Présentation du mode ECB

Données. On considère un cryptosystème par blocs de taille $n = 4$ bits, fonctionnant en mode ECB. La fonction de chiffrement est une **permutation** π définie par :

$$b_1 b_2 b_3 b_4 \longrightarrow b_{\pi(1)} b_{\pi(2)} b_{\pi(3)} b_{\pi(4)} \qquad \text{avec} \quad \pi = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 1 \end{pmatrix}$$

Le texte clair est : $m = \texttt{101100010100101}$

1.2 Question 1 — Fonctionnement du mode ECB (pseudo-code)

En mode ECB, chaque bloc est chiffré **indépendamment** avec la même clé :

Listing 1 – Pseudo-code du mode ECB

```
Chiffrement ECB :
  Diviser m en blocs B_1, B_2, ..., B_k de taille n (bourrer le
     dernier si necessaire)
  Pour i de 1 a k :
    C_i = E_K(B_i)   // application de la permutation pi
  Retourner C = C_1 || C_2 || ... || C_k

Dechiffrement ECB :
  Diviser C en blocs C_1, C_2, ..., C_k de taille n
  Pour i de 1 a k :
    B_i = D_K(C_i)   // application de pi^{-1}
  Retourner m = B_1 || B_2 || ... || B_k
```

1.3 Question 2 — Découpage en blocs

Le message $m = \texttt{101100010100101}$ comporte 15 bits. On le découpe en blocs de 4 bits, en bourrant le dernier avec un zéro :

Bloc	B_1	B_2	B_3	B_4
Bits	1011	0001	0100	1010

Table 1 – Découpage de m en blocs de 4 bits (bourrage d'un zéro)

1.4 Question 3 & 4 — Chiffrement ECB et texte chiffré final

La permutation $\pi = (2\,3\,4\,1)$ décale les positions d'un rang vers la gauche (le bit en position 1 va en position 4) :

Bloc clair	Positions $(b_1b_2b_3b_4)$	Après π $(b_2b_3b_4b_1)$	Bloc chiffré
$B_1 = \texttt{1011}$	1, 0, 1, 1	0, 1, 1, 1	$C_1 = \texttt{0111}$
$B_2 = \texttt{0001}$	0, 0, 0, 1	0, 0, 1, 0	$C_2 = \texttt{0010}$
$B_3 = \texttt{0100}$	0, 1, 0, 0	1, 0, 0, 0	$C_3 = \texttt{1000}$
$B_4 = \texttt{1010}$	1, 0, 1, 0	0, 1, 0, 1	$C_4 = \texttt{0101}$

Table 2 – Chiffrement ECB bloc par bloc

Texte chiffré final : $C = \texttt{0111 0010 1000 0101}$

1.5 Question 5 — Déchiffrement et vérification

La permutation inverse $\pi^{-1} = (4\,1\,2\,3)$ (rotation d'un rang vers la droite) est appliquée à chaque bloc chiffré :

Bloc chiffré	Après π^{-1} $(b_4b_1b_2b_3)$	Bloc clair retrouvé
$C_1 = \texttt{0111}$	1, 0, 1, 1	$B_1 = \texttt{1011}$
$C_2 = \texttt{0010}$	0, 0, 0, 1	$B_2 = \texttt{0001}$
$C_3 = \texttt{1000}$	0, 1, 0, 0	$B_3 = \texttt{0100}$
$C_4 = \texttt{0101}$	1, 0, 1, 0	$B_4 = \texttt{1010}$

Table 3 – Déchiffrement ECB — vérification

On retrouve bien $m = \texttt{101100010100101}$ (en supprimant le bit de bourrage). ✓

1.6 Question 6 — Redondance dans le texte chiffré

Si le texte clair est composé de blocs identiques `1010`, alors chaque bloc produit le même chiffré `0101`. **La redondance du clair est directement visible dans le chiffré.** C'est la principale faiblesse du mode ECB : des blocs clairs identiques donnent des blocs chiffrés identiques, ce qui révèle des patterns statistiques à l'attaquant.

1.7 Question 7 — Permutation de l'ordre des blocs chiffrés

En mode ECB, chaque bloc chiffré peut être déchiffré **indépendamment**. Ainsi, si l'on permute l'ordre des blocs chiffrés, **chaque bloc se déchiffre correctement**, mais le message reconstruit sera incohérent. ECB est donc vulnérable aux attaques par *cut-and-paste* (réorganisation de blocs).

1.8 Question 8 — Bilan de sécurité du mode ECB

ECB est le mode le plus simple mais aussi le **plus faible** :

- Blocs identiques ⇒ chiffrés identiques : les patterns sont préservés (cf. l'exemple classique de l'image « ECB Penguin ») ;
- Absence de propagation : chaque bloc est traité de façon totalement indépendante ;
- Vulnérable aux attaques par rejeu et réorganisation de blocs.

ECB est approprié uniquement pour chiffrer des **données courtes et non répétitives**, comme le chiffrement d'une clé de session unique ou de blocs aléatoires.

2 Exercice 2 — Mode CBC (*Cipher Block Chaining*)

2.1 Présentation du mode CBC

On réutilise la même permutation $\pi = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 1 \end{pmatrix}$ et le même texte clair $m =$ `101100010100101`, mais en mode **CBC**.

2.2 Question 1 — Fonctionnement du mode CBC (pseudo-code)

En mode CBC, chaque bloc clair est d'abord **XORé** avec le bloc chiffré précédent avant d'être chiffré. Le vecteur d'initialisation $IV = c_0$ est utilisé pour le premier bloc :

Listing 2 – Pseudo-code du mode CBC

```
Chiffrement CBC :
  c_0 = IV   (vecteur d'initialisation)
  Pour i de 1 a k :
    C_i = E_K(B_i XOR C_{i-1})
  Retourner C = C_1 || C_2 || ... || C_k

Dechiffrement CBC :
  c_0 = IV
  Pour i de 1 a k :
    B_i = D_K(C_i) XOR C_{i-1}
  Retourner m = B_1 || B_2 || ... || B_k
```

2.3 Question 2 — Découpage en blocs

Identique à l'exercice 1 (bourrage d'un zéro sur le dernier bloc) :

Bloc	B_1	B_2	B_3	B_4
Bits	1011	0001	0100	1010

Table 4 – Découpage de m en blocs de 4 bits

On pose $IV = C_0 =$ `0000`.

2.4 Question 3 & 4 — Chiffrement CBC et texte chiffré final

Étape	B_i	C_{i-1}	$B_i \oplus C_{i-1}$	$C_i = \pi(B_i \oplus C_{i-1})$
$i = 1$	1011	0000	1011	0111
$i = 2$	0001	0111	0110	1100
$i = 3$	0100	1100	1000	0001
$i = 4$	1010	0001	1011	0111

Table 5 – Chiffrement CBC bloc par bloc ($IV =$ `0000`)

Texte chiffré final : $C =$ `0111 1100 0001 0111`

2.5 Question 5 — Déchiffrement et vérification

Étape	C_i	$\pi^{-1}(C_i)$	C_{i-1}	$B_i = \pi^{-1}(C_i) \oplus C_{i-1}$
$i = 1$	0111	1011	0000	1011
$i = 2$	1100	0110	0111	0001
$i = 3$	0001	1000	1100	0100
$i = 4$	0111	1011	0001	1010

Table 6 – Déchiffrement CBC — vérification

On retrouve bien $m =$ 101100010100101 (en supprimant le bit de bourrage). ✓

2.6 Question 6 — Redondance dans le texte chiffré

En CBC, si le texte clair contient des blocs identiques (1011), les blocs chiffrés correspondants seront **différents**, car chaque chiffrement dépend du bloc précédent. Ainsi, **la redondance n'est pas propagée dans le chiffré** : CBC masque efficacement les patterns du texte clair.

2.7 Question 7 — Permutation de l'ordre des blocs chiffrés

En CBC, si l'on permute des blocs chiffrés :

— Le déchiffrement d'un bloc C_i permis reste possible (il produit un résultat, mais incorrect) ;

— Les blocs adjacents sont affectés car le déchiffrement de C_i dépend de C_{i-1}.

Contrairement à ECB, la réorganisation détruit l'intégrité du message : on ne peut pas reconstituer le clair original.

2.8 Question 8 — Bilan de sécurité du mode CBC

CBC offre une sécurité bien supérieure à ECB :

— Les blocs clairs identiques produisent des blocs chiffrés différents ;

— Le chiffrement est séquentiel (non parallélisable à l'émission) ;

— Nécessite un IV aléatoire et imprévisible pour chaque message.

CBC est approprié pour le chiffrement de fichiers, de flux de données et est utilisé dans de nombreux protocoles (SSL/TLS, IPSec).

2.9 Question 9 — Propagation d'erreur

Si une erreur survient dans le **premier bloc chiffré** C_1 :

— Le déchiffrement de C_1 donnera un bloc clair B_1 **corrompu** (tous ses bits sont potentiellement altérés) ;

— Le déchiffrement de C_2 utilise C_1 corrompu pour le XOR : B_2 sera également **corrompu** ;

— À partir de C_3, le déchiffrement utilise C_2 (non corrompu en entrée de π) : les blocs $B_3, B_4, \ldots$ sont **intacts**.

En résumé : en CBC, une erreur dans un bloc chiffré n'affecte que **deux blocs clairs consécutifs**.

2.10 Question 10 — Applications de CBC

CBC est adapté aux applications où :

— On chiffre des **fichiers ou messages longs** dont les patterns doivent être masqués ;

— On peut tolérer une propagation d'erreur limitée à deux blocs ;

— Le chiffrement séquentiel est acceptable (données stockées, non temps-réel).

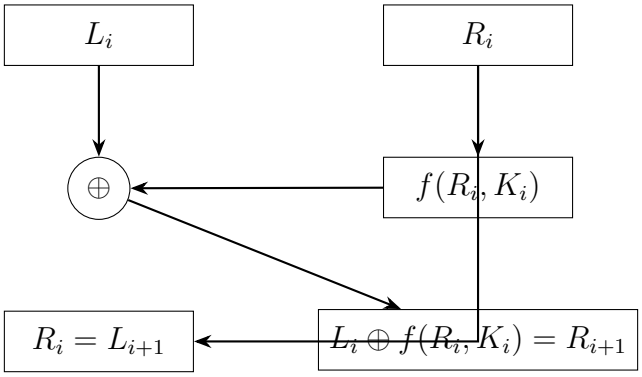
Exemples : chiffrement de disques, protocoles SSL/TLS (historiquement), intégrité de fichiers.

3 Exercice 3 — Schéma de Feistel (2 tours)

3.1 Principe du schéma de Feistel

Le schéma de Feistel est un système de chiffrement **itératif** : le bloc de données est divisé en deux moitiés, et à chaque tour, une fonction f (potentiellement non inversible) est appliquée à une moitié pour modifier l'autre. La symétrie du schéma garantit que le déchiffrement suit la même architecture que le chiffrement, en inversant simplement l'ordre des sous-clés.

Schéma d'un tour



3.2 Algorithme implémenté (Python)

L'algorithme suit les étapes suivantes :

1. Créer la liste des caractères du texte clair ;
2. Convertir chaque caractère en ASCII, puis en représentation binaire sur 8 bits ;
3. Diviser la chaîne binaire en deux moitiés L_1 et R_1 ;
4. Générer deux clés binaires aléatoires K_1 et K_2, chacune de longueur égale à une moitié du texte ;
5. Appliquer 2 tours de Feistel pour chiffrer, puis les mêmes tours en ordre inverse pour déchiffrer.

Tour 1 (chiffrement)

$$f_1 = R_1 \oplus K_1$$
$$R_2 = f_1 \oplus L_1 = R_1 \oplus K_1 \oplus L_1 \qquad L_2 = R_1$$

Tour 2 (chiffrement)

$$f_2 = R_2 \oplus K_2$$
$$R_3 = f_2 \oplus L_2 = R_2 \oplus K_2 \oplus L_2 \qquad L_3 = R_2$$

Le texte chiffré est la concaténation $C = L_3 \parallel R_3$.

Déchiffrement

Le déchiffrement applique les mêmes clés en **ordre inverse** (K_2 puis K_1) :

$$f'_2 = L_3 \oplus K_2 \qquad R'_2 = f'_2 \oplus R_3 \qquad L'_2 = L_3$$
$$f'_1 = L'_2 \oplus K_1 \qquad R'_1 = f'_1 \oplus R'_2 \qquad L'_1 = L'_2$$

On reconstruit le texte clair en concaténant $L'_1 \parallel R'_1$ et en convertissant les blocs binaires en ASCII.

3.3 Code Python

Listing 3 – Implémentation du schéma de Feistel en Python

```python
import random

def text_to_bin(text):
    """Convertit un texte en chaine binaire (8 bits par caractere)."""
    return ''.join(format(ord(c), '08b') for c in text)

def bin_to_text(bin_str):
    """Convertit une chaine binaire en texte (blocs de 8 bits)."""
    chars = [bin_str[i:i+8] for i in range(0, len(bin_str), 8)]
    return ''.join(chr(int(c, 2)) for c in chars)

def xor_bits(a, b):
    """XOR bit a bit de deux chaines binaires de meme longueur."""
    return ''.join('1' if x != y else '0' for x, y in zip(a, b))

def generate_key(length):
    """Genere une cle binaire aleatoire de longueur donnee."""
    return ''.join(random.choice('01') for _ in range(length))

def feistel_encrypt(plaintext):
```

```python
    """Chiffrement Feistel a 2 tours."""
    # Etape 1 & 2 : conversion en binaire
    binary = text_to_bin(plaintext)

    # Bourrage pour obtenir une longueur paire
    if len(binary) % 2 != 0:
        binary += '0'

    half = len(binary) // 2

    # Etape 3 : division en deux moities
    L1 = binary[:half]
    R1 = binary[half:]

    # Etape 4 : generation des cles
    K1 = generate_key(half)
    K2 = generate_key(half)

    print(f"Texte clair   : {plaintext}")
    print(f"Binaire       : {binary}")
    print(f"L1 = {L1}")
    print(f"R1 = {R1}")
    print(f"K1 = {K1}")
    print(f"K2 = {K2}")

    # --- Tour 1 ---
    f1 = xor_bits(R1, K1)
    R2 = xor_bits(f1, L1)
    L2 = R1
    print(f"\n[Tour 1]")
    print(f"f1 = R1 XOR K1 = {f1}")
    print(f"L2 = R1        = {L2}")
    print(f"R2 = f1 XOR L1 = {R2}")

    # --- Tour 2 ---
    f2 = xor_bits(R2, K2)
    R3 = xor_bits(f2, L2)
    L3 = R2
    print(f"\n[Tour 2]")
    print(f"f2 = R2 XOR K2 = {f2}")
    print(f"L3 = R2        = {L3}")
    print(f"R3 = f2 XOR L2 = {R3}")

    ciphertext_bin = L3 + R3
    print(f"\nTexte chiffre (binaire) : {ciphertext_bin}")

    return ciphertext_bin, K1, K2

def feistel_decrypt(ciphertext_bin, K1, K2):
    """Dechiffrement Feistel a 2 tours (cles en ordre inverse)."""
```

```python
    half = len(ciphertext_bin) // 2
    L3 = ciphertext_bin[:half]
    R3 = ciphertext_bin[half:]

    print(f"\n=== Dechiffrement ===")
    print(f"L3 = {L3}")
    print(f"R3 = {R3}")

    # --- Tour 2 inverse (avec K2) ---
    f2 = xor_bits(L3, K2)
    R2 = xor_bits(f2, R3)
    L2 = L3
    print(f"\n[Tour 2 inverse]")
    print(f"f2' = L3 XOR K2 = {f2}")
    print(f"L2' = L3        = {L2}")
    print(f"R2' = f2' XOR R3= {R2}")

    # --- Tour 1 inverse (avec K1) ---
    f1 = xor_bits(L2, K1)
    R1 = xor_bits(f1, R2)
    L1 = L2
    print(f"\n[Tour 1 inverse]")
    print(f"f1' = L2' XOR K1 = {f1}")
    print(f"L1' = L2'        = {L1}")
    print(f"R1' = f1' XOR R2'= {R1}")

    recovered_bin = L1 + R1
    # Supprimer le bit de bourrage si necessaire
    if len(recovered_bin) % 8 != 0:
        recovered_bin = recovered_bin[:-1]

    recovered_text = bin_to_text(recovered_bin)
    print(f"\nTexte dechiffre (binaire) : {recovered_bin}")
    print(f"Texte dechiffre           : {recovered_text}")
    return recovered_text

# --- Programme principal ---
plaintext = "Bonjour"
print("=" * 50)
print("=== Chiffrement Feistel ===")
ciphertext_bin, K1, K2 = feistel_encrypt(plaintext)

print("\n" + "=" * 50)
recovered = feistel_decrypt(ciphertext_bin, K1, K2)

print("\n" + "=" * 50)
print(f"Verification : texte original = {plaintext}")
print(f"               texte recupere  = {recovered}")
print(f"               OK ? {plaintext == recovered}")
```

3.4 Exemple d'exécution

L'exécution du programme sur le texte `"Bonjour"` produit une sortie de la forme suivante (les clés étant aléatoires, les valeurs changent à chaque exécution) :

Listing 4 – Exemple de sortie (valeurs indicatives)

```
==================================================
=== Chiffrement Feistel ===
Texte clair   : Bonjour
Binaire       :
   0100001001101111011011100110101001110101011101010111 0010
L1 = 01000010011011110110111001101010
R1 = 01110101011101010111001000000000   (+ bourrage)
K1 = [cle aleatoire 32 bits]
K2 = [cle aleatoire 32 bits]

[Tour 1]
f1 = R1 XOR K1 = ...
L2 = R1        = ...
R2 = f1 XOR L1 = ...

[Tour 2]
f2 = R2 XOR K2 = ...
L3 = R2        = ...
R3 = f2 XOR L2 = ...

Texte chiffre (binaire) : [64 bits]

==================================================
=== Dechiffrement ===
[Tour 2 inverse] ...
[Tour 1 inverse] ...

Texte dechiffre : Bonjour
==================================================
Verification : OK ? True
```

3.5 Analyse du schéma de Feistel

Quelques propriétés importantes à retenir :

- **Inversibilité garantie :** quelle que soit la fonction f utilisée, le schéma est toujours inversible (la fonction f n'a pas besoin d'être bijective) ;
- **Même architecture chiffrement/déchiffrement :** seul l'ordre des sous-clés change, ce qui simplifie l'implémentation matérielle ;
- **Nombre de tours :** avec seulement 2 tours et une fonction $f = \text{XOR}$, la sécurité reste faible (connaissance de la structure suffit). Les algorithmes réels (DES : 16 tours, Blowfish : 16 tours) utilisent bien plus de tours et des fonctions f complexes ;
- **Base du DES :** le DES (Data Encryption Standard) repose intégralement sur ce schéma.

Conclusion

Ce TP a permis d'étudier et d'implémenter les principaux modes opératoires des chiffrements par blocs, ainsi que le schéma de Feistel. Les points clés retenus sont les suivants :

— Le **mode ECB** est simple mais dangereux : il préserve les patterns du texte clair dans le chiffré et est vulnérable aux attaques par réorganisation de blocs. Son usage est déconseillé pour tout message contenant des répétitions ;

— Le **mode CBC** corrige les faiblesses d'ECB grâce au chaînage : les blocs clairs identiques produisent des blocs chiffrés différents. En contrepartie, une erreur dans un bloc chiffré se propage sur deux blocs lors du déchiffrement ;

— Le **schéma de Feistel** fournit un cadre élégant pour construire des chiffrements par blocs inversibles à partir de fonctions non inversibles. Il constitue la base d'algorithmes historiquement importants comme le DES. L'implémentation en Python confirme la symétrie entre chiffrement et déchiffrement.